Claim Outcome Prediction: Project Explained

An end-to-end machine learning mini build: data, model, evaluation and web API

1. What this project is, in one minute

Insurance claims are submitted by healthcare providers to insurance companies (called payers). Each claim eventually ends in one of three outcomes:

- **Paid**: the payer pays the claim in full.
- **Partially_Paid**: the payer pays only part of the amount.
- **Denied**: the payer refuses to pay.

This project builds a small system that **predicts the outcome of a claim before it happens**, using only the claim's details (payer, provider, procedure code, diagnosis code, billed amount, patient age). The prediction is made by a machine learning model, and the model is exposed through a web API so that any other program can send in a claim and get a prediction back.

Why it matters: if a claim is likely to be denied, a billing team can fix it or prepare an appeal *before* submission, which saves time and money.

No healthcare knowledge is needed. Payer, procedure code and diagnosis code are treated as ordinary categorical columns in a table, as in any classification problem.

2. What was built (the deliverables)

File	What it does
`generate_data.py`	Creates a simulated dataset of 450 historical claims and saves it as `claims.csv`.
`features.py`	Defines the feature engineering (new helpful columns) in a way that avoids data leakage.
`train.py`	Splits the data, trains the model, evaluates it, and saves the trained pipeline to `model.joblib`.

File	What it does
`app.py`	The web API (FastAPI). Exposes `POST /predict` and the bonus `GET /appeal-guidance`.
`appeal.py`	Bonus: finds the most relevant appeal-guidance text for a denial reason using keyword matching.
`test_api.py`	Automated tests that check the API behaves correctly, including on bad input.
`requirements.txt`	The list of Python libraries needed.
`README.md`	Short run instructions and results.
`metrics.txt`, `model.joblib`, `claims.csv`	Files generated when the scripts are run.

3. The big picture: how the pieces fit together

The project runs as a simple chain:

1. **Generate data**: `generate_data.py` writes `claims.csv`.
2. **Train**: `train.py` reads `claims.csv`, splits it, trains a model, prints the evaluation, and writes `model.joblib`.
3. **Serve**: `app.py` loads `model.joblib` once at start-up and answers prediction requests.
4. **Use**: a client (a script, a website, `curl`) sends a claim as JSON and receives the predicted status and probabilities.

Important design choice: the **preprocessing and the model are saved together as a single pipeline**. This guarantees that a new claim sent to the API is transformed in exactly the same way as the training data was, so the two can never get out of sync.

4. Step 0: The dataset

4.1 Why simulated data

The task allows (and expects) the data to be generated by a script. The goal is not data sourcing but building a correct ML pipeline.

4.2 Columns

Each row is one past claim:

Column	Meaning	Example
id	Row number (not used for learning)	1
payer	The insurance company	Aetna, Cigna, UnitedHC, BlueCross
provider	The doctor who submitted it	Dr.Rao
cpt_code	Procedure code (treated as a category)	99213
diagnosis_code	Diagnosis code (treated as a category)	M54.5
billed_amount	Amount charged	220.00
patient_age	Age of the patient	45
status	**The target to predict**: Paid, Partially_Paid or Denied	Paid

4.3 Deliberate class imbalance

The dataset is intentionally imbalanced to mirror real claims data. The generated mix was:

Status	Share
Paid	66.4%
Partially_Paid	22.7%
Denied	10.9%

This is close to the requested 70/20/10. Because the data is generated randomly, the shares vary slightly from run to run (a fixed random seed of 42 makes the result repeatable).

4.3 Hidden patterns planted in the data

If the data were pure noise, no model could learn anything. So the generator builds in realistic tendencies that the model is expected to discover:

- Some payers deny more often than others (for example Cigna is set higher than Aetna).
- Some procedure codes are more likely to be only partially paid (for example 99215).
- Claims billed far above the typical amount for their procedure are more likely to be denied.
- Older patients (over 65) are somewhat more likely to be partially paid.
- Billed amounts follow a realistic spread around a typical price per procedure.

The signal is deliberately modest, which is why the final model performance is modest (see section 8).

5. Step 1: Preparing the data

5.1 Stratified train/test split

The data is divided into:

- a **training set (80%, 360 rows)**: what the model learns from;
- a **test set (20%, 90 rows)**: kept hidden until the end, to check the model honestly on rows it has never seen.

Because Denied claims are rare, a plain random split could by chance put most of them on one side. So a **stratified split** is used: each side keeps the same class mix as the whole dataset.

	Paid	Partially_Paid	Denied
Training set	66.4%	22.8%	10.8%
Test set	66.7%	22.2%	11.1%

The mixes are almost identical, which confirms the split worked.

5.2 Feature engineering

Feature engineering means creating new columns that are not in the raw data but may help the model. The task asked for at least two. This project creates five:

New feature	How it is calculated	Why it may help
`payer_denial_rate`	Share of that payer's training claims that were Denied	Tells the model how strict each payer is.
`provider_denial_rate`	Share of that provider's training claims that were Denied	Some providers submit more error-prone claims.
`cpt_partial_rate`	Share of that procedure code's training claims that were Partially_Paid	Some procedures are often paid only in part.
`billed_vs_cpt_median`	Billed amount divided by the typical (median) billed amount for that procedure	Spots unusually high charges.
`is_senior`	1 if age is 65 or more, else 0	Captures an age effect with a simple flag.

The raw columns are also kept, so the model sees both raw and engineered information.

5.3 Smoothing

A payer or provider with very few claims could get an extreme rate by luck (for example 1 denial out of 2 claims is 50%). To avoid trusting such small samples, the rates are **smoothed**: they are pulled toward the overall average, and the effect is stronger for categories with fewer rows. The smoothing strength is 10 (it behaves like adding 10 imaginary average claims to every category).

5.4 Data leakage: the most important concept

Data leakage means information from the test set sneaks into training, which makes results look better than they would be in real life.

Example of the mistake to avoid: calculating `payer_denial_rate` using the *whole* dataset (training plus test) before splitting. The test rows' answers would then influence a feature the model trains on.

How this project prevents it:

1. The data is **split first**.
2. The feature calculations live inside a custom class, `ClaimFeatures` (in `features.py`), which follows the scikit-learn pattern of a `fit` step and a `transform` step.
3. `fit` is only ever run on the **training rows**. It records the rates and medians.
4. `transform` only **looks up** those stored numbers. It is applied to training rows, test rows, and brand-new API requests alike.
5. If a payer or provider was never seen in training, a fallback value (the overall average) is used instead of crashing.

Because this class is part of the same pipeline as the model, the leakage protection is automatic and cannot be forgotten later.

6. Step 2: Training the model

6.1 The pipeline

The saved object is one scikit-learn `Pipeline` with three stages:

1. **ClaimFeatures**: adds the five engineered features.
2. **ColumnTransformer**: converts text categories (payer, provider, cpt_code, diagnosis_code) into numbers using one-hot encoding (one 0/1 column per category value), and passes numeric columns through unchanged. Unknown categories seen later are ignored safely rather than raising an error.
3. **RandomForestClassifier**: the actual model.

6.2 Why a Random Forest

A Random Forest is many decision trees voting together. It handles a mix of categorical and numeric features well, needs little tuning, copes with small datasets, and gives probability estimates, which the API returns.

6.3 Handling the imbalance

The model is trained with `class_weight="balanced_subsample"`. This makes mistakes on rare classes (Denied, Partially_Paid) count more during training, so the model does not simply learn to always answer "Paid".

Other settings: 300 trees, at least 3 rows per leaf (to reduce over-fitting), and a fixed random seed for repeatability.

6.4 Saving the model

After training, the whole pipeline is saved with `joblib` into `model.joblib`. The API loads this file at start-up, so no retraining is needed to serve predictions.

7. Step 3: Evaluating the model properly

7.1 Why accuracy is not enough

If 2 out of 3 claims are Paid, a useless model that always answers "Paid" scores about 67% accuracy while never catching a single Denied claim. Since Denied claims are the ones that matter most, accuracy alone hides the real failure.

7.2 The metrics used

For each class separately:

- **Precision**: of the claims the model *labelled* as this class, how many truly were. (How trustworthy are its alarms?)
- **Recall**: of the claims that *actually were* this class, how many the model caught. (How many did it miss?)
- **F1-score**: a single number that balances precision and recall.
- **Support**: how many test rows belong to the class.

Plus a **confusion matrix**: a table showing, for each actual class, what the model predicted.

7.3 Results on the 90 held-out test claims

Class	Precision	Recall	F1	Support
Paid	0.686	0.800	0.738	60

Class	Precision	Recall	F1	Support
Partially_Paid	0.182	0.100	0.129	20
Denied	0.222	0.200	0.211	10

Overall accuracy: 0.578.

Confusion matrix (rows are the actual class, columns are the prediction):

	Predicted Paid	Predicted Partially_Paid	Predicted Denied
Actual Paid	48	7	5
Actual Partially_Paid	16	2	2
Actual Denied	6	2	2

7.4 How to read these results honestly

- The model is **best at Paid** (the majority class) and **weak on the two minority classes**.
- **Partially_Paid is the weakest**: only 2 of 20 were caught (recall 0.10). Denied is next: 2 of 10 caught (recall 0.20). Most minority claims are mistaken for Paid.
- Accuracy (0.578) is actually **below** the 0.667 that an always-Paid guesser would get. This is a clear example of why the task insists on per-class metrics, and it should be stated plainly rather than hidden.
- Why the performance is low: the dataset is small (450 rows), the planted signal is weak and noisy, and the test set contains only 10 Denied claims, so one or two claims change the numbers a lot. These figures should be treated as a working baseline, not as proof of real-world performance.

7.5 Ideas to improve the weak classes

- Collect more data, and richer features such as prior-authorization status or past denial reasons.
- Tune the decision threshold for Denied, so the model flags a claim as Denied at a lower probability. This trades some precision for higher recall.
- Oversample the minority classes (for example with SMOTE) or increase their weights further.
- Replace the single split with stratified cross-validation, giving more stable estimates.

- Try other models such as gradient boosting (XGBoost or LightGBM) and tune their settings.

8. Step 4: Serving the model through an API

8.1 What an API is here

The API is a small web service. A client sends a claim as JSON to the address `/predict` and gets the prediction back as JSON. It is built with **FastAPI** and run with **uvicorn**.

8.2 Example request

```
{
  "payer": "Aetna",
  "provider": "Dr. Rao",
  "cpt_code": "99213",
  "diagnosis_code": "M54.5",
  "billed_amount": 220.00,
  "patient_age": 45
}
```

8.3 Example response

```
{
  "predicted_status": "Paid",
  "probabilities": {
    "Paid": 0.81,
    "Partially_Paid": 0.14,
    "Denied": 0.05
  }
}
```

The probabilities add up to 1, and the predicted status is the class with the highest probability. (The numbers above are the format example from the task; real values come from the model.)

8.4 Input validation and error handling

The request is checked before it reaches the model. The rules are:

- All six fields are required.

- Text fields must not be empty.
- `billed_amount` must be a number greater than 0.
- `patient_age` must be a whole number between 0 and 120.

If anything is wrong (a missing field, or `billed_amount` sent as the text "abc"), the API returns a clear **HTTP 400** error listing which field has which problem, instead of crashing. FastAPI normally answers such errors with code 422, so a custom handler converts it to 400 as the task requires.

8.5 Small practical details

- The provider name `Dr. Rao` (with a space) is converted to `Dr.Rao`, which is how it appears in the training data.
- A numeric procedure code such as 99213 is accepted and converted to text.
- A payer never seen in training does not crash the service. It simply falls back to average values.

8.6 Automated tests

`test_api.py` checks four situations, and all four pass:

1. A valid claim returns status 200 and probabilities that sum to 1.
2. A claim with a missing field returns 400.
3. A claim with text in `billed_amount` returns 400.
4. A claim with an unseen payer still returns 200.

9. Bonus: appeal guidance

When a claim is denied, the billing team often writes an appeal. The bonus part provides ready-made guidance text for common denial reasons.

- `appeal.py` holds 7 short guidance snippets: missing authorization, eligibility lapse, duplicate claim, timely filing, coding error, medical necessity, and out of network.
- Each snippet has a list of keywords. The function `get_appeal_guidance(denial_reason)` counts how many keywords appear in the given reason (with an extra boost for an exact reason name) and returns the best match.
- If nothing matches, it returns a message to escalate to a human biller.

- It is available through `GET /appeal-guidance?denial_reason=missing_auth`.

Simple keyword matching is used on purpose, as the task allows. No embeddings or LLM are needed.

9.1 Reducing hallucination if an LLM later writes the appeal letter

A language model may invent facts. To limit this:

1. Give the model the retrieved snippet as its only permitted source of guidance.
2. Provide patient, date, amount and claim details as structured fields, and forbid inventing them.
3. Tell it to answer "insufficient information" rather than guess missing details.
4. Use a fixed letter template where the model fills only specific slots.
5. Use a low temperature and structured output validated against a schema.
6. After generation, automatically check that every number, date and policy reference in the draft appears in the inputs.
7. Forbid citing laws or policy numbers that are not in the retrieved text.
8. Require a human reviewer to approve every letter before it is sent.

10. How to run everything (commands)

```
# 1. Create and activate a virtual environment, install libraries
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# 2. Generate the dataset (creates claims.csv)
python generate_data.py

# 3. Train and evaluate (creates model.joblib and metrics.txt)
python train.py

# 4. Start the API
uvicorn app:app --reload --port 8000

# 5. Send a prediction request (in a second terminal)
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"payer":"Aetna","provider":"Dr.
```

```
Rao","cpt_code":"99213","diagnosis_code":"M54.5","billed_amount":220.0,"patie
nt_age":45}'

# 6. Send a bad request (expect HTTP 400)
curl -i -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"payer":"Aetna","billed_amount":"abc"}'

# 7. Try the bonus endpoint
curl "http://127.0.0.1:8000/appeal-guidance?denial_reason=missing_auth"

# 8. Run the tests
pytest -q
```

You can also open `http://127.0.0.1:8000/docs` in a browser for an interactive page where requests can be tried without `curl`.

11. Assumptions and shortcuts (because of the time limit)

- **Synthetic data**: the dataset is simulated with hand-chosen effects, so the results describe this simulation, not real payer behaviour.
- **Single split**: one 80/20 split on 450 rows. The Denied class has only 10 test rows, so its metrics are very noisy. Cross-validation would be more reliable.
- **Target-rate features include each row's own label**: when the rates are computed on the training set, every training row contributes its own label to its category's rate. This can make the model slightly over-trust these features on training data. Smoothing reduces the effect, and the cleaner fix is out-of-fold target encoding.
- **No hyperparameter tuning**: sensible default-style settings were used.
- **Minimal API**: no authentication, logging or monitoring, and the model is loaded once at start-up.
- **Provider name format** is normalised by removing spaces.

12. Glossary

Term	Plain meaning
Payer	The insurance company that pays claims.

Term	Plain meaning
Provider	The doctor or clinic that submits the claim.
CPT code	A code for the medical procedure performed (here, just a category).
Diagnosis code	A code for the patient's condition (here, just a category).
Class / label	One of the possible outcomes: Paid, Partially_Paid, Denied.
Imbalanced data	Some classes are far rarer than others.
Stratified split	A split that keeps each class's proportion the same on both sides.
Feature	An input column the model learns from.
Feature engineering	Creating new, useful columns from existing ones.
Data leakage	Test-set information accidentally influencing training.
Pipeline	Preprocessing steps and the model bundled as one object.
One-hot encoding	Turning a category into several 0/1 columns.
Precision / Recall / F1	Measures of how trustworthy, how complete, and how balanced a class's predictions are.
Confusion matrix	Table of actual versus predicted classes.
API / endpoint	A web address that other programs call to use the model.

13. Summary

This project takes a table of past insurance claims, splits it fairly, creates five leakage-safe features, trains a class-weighted Random Forest, evaluates it with per-class precision, recall, F1 and a confusion matrix, saves the full pipeline, and serves it through a validated FastAPI endpoint with tests. A keyword-based appeal-guidance feature is included as a bonus. The model currently performs well on Paid claims and poorly on the rare classes,

and the document explains why and lists concrete next steps. The main value of the project is the correct, reproducible process: split first, learn features from training data only, measure with the right metrics, and serve with safe input handling.